



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления (ИУ)»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии (ИУ7)»

ОТЧЕТ

Лабораторная работа №3

по курсу «Проектирование экспертных систем»

на тему: «Обратный поиск в глубину от цели в графах И/ИЛИ»

Студент ИУ7-32М
(Группа)

(Подпись, дата)

К.Э. Ковалец
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

З.Н. Русакова
(И. О. Фамилия)

2024 г.

Содержание

1	Основная часть	3
1.1	Описание метода	3
1.2	Постановка задачи	3
1.3	Алгоритм поиска потомков	4
1.4	Алгоритм возврата	4
1.5	Алгоритм разметки	4
1.6	Результаты работы программы	5
2	Текст программы	7

1 Основная часть

1.1 Описание метода

Обратный поиск в глубину — это метод, который используется для нахождения пути от заданной цели к начальным вершинам в графе И/ИЛИ. В процессе работы используются следующие списки:

- стек открытых вершин;
- список закрытых вершин;
- список запрещенных вершин;
- список открытых правил;
- список закрытых правил;
- список запрещенных правил.

1.2 Постановка задачи

Необходимо разработать класс метода поиска в глубину, полями которого являются указанные списки, целевая вершина, а также два флага: `solution_flg` — есть решение и `no_solution_flg` — нет решения, имеющие значения по умолчанию `false`. В конструктор класса передается список базы правил. Целевая вершина и массив заданных входных вершин задаются при запуске метода поиска. Входные вершины записываются в закрытые вершины, целевая вершина помещается в голову стека (список открытых вершин). Все вершины помечаются флагами, равными нулю.

Разрабатываются четыре метода:

- «поиск потомков» — определяет первое правило, раскрывающее текущую подцель, и возвращает, было ли найдено правило;
- «поиск» (основной метод) — вызывает все методы алгоритма, работает в цикле `while`, пока флаги равны `false`;
- «возврат» (`back tracking`) — отсекает запрещённые вершины и правила;
- «разметка» — формирует список закрытых правил, составляющих дерево решения, и список закрытых (доказанных) вершин.

1.3 Алгоритм поиска потомков

Текущая подцель выбирается из головы стека открытых вершин. В цикле по базе правил определяется первое правило, выходная вершина которого совпадает с подцелью (раскрывает эту вершину). Номер правила записывается в голову стека открытых правил. Затем определяются новые подцели и записываются в стек открытых вершин. Для этого необходимо определить, какие вершины выбранного правила из входных не входят в закрытые. Эти вершины добавляются в голову стека открытых вершин. Если все входные вершины правила попали в закрытые, то новых подцелей нет и вызывается процедура разметки.

1.4 Алгоритм возврата

Если потомков не найдено, функция возвращает ноль. Это означает, что текущая подцель должна быть объявлена как запрещённая вершина, а само правило — как запрещённое. Необходимо удалить текущую подцель из стека открытых вершин (из головы), объявить правило из головы стека запрещённым, удалить его и перенести в запрещённые. Также необходимо удалить из стека все вершины, которые являлись входными вершинами найденного запрещённого правила. После этого снова вызывается метод поиска потомков для нахождения альтернативного пути. Если альтернативы нет, то снова происходит возврат. Если в стеке открытых вершин осталась только одна целевая после ненахождения нужного правила, то решение не найдено (`no_solution_flg = true`).

1.5 Алгоритм разметки

Этот алгоритм используется для построения дерева решения и закрытия вершин и правил. Он работает путём последовательного подъема снизу-вверх по дереву.

Основные шаги алгоритма:

- извлечение правила из списка открытых правил и добавление его в список закрытых правил;
- извлечение вершины из списка открытых вершин и добавление ее в список закрытых вершин;

- проверка, является ли текущая вершина целевым узлом. Если да, то найдено решение (`solution_flg = true`);
- если текущая вершина не является целевой, то необходимо вернуться к последнему открытому узлу и последнему открытому правилу и проверить, соответствует ли текущее правило текущему узлу. Если нет, то алгоритм завершается.

1.6 Результаты работы программы

Следующими цветами обозначены отдельные группы правил и вершин:

- зеленым — целевая вершина графа;
- синим — входные вершины графа;
- фиолетовым — закрытые вершина и правила графа, не включая входные вершины;
- оранжевым — запрещенные вершины и правила графа.

Результаты работы программы приведены на рисунках 1.1–1.2.

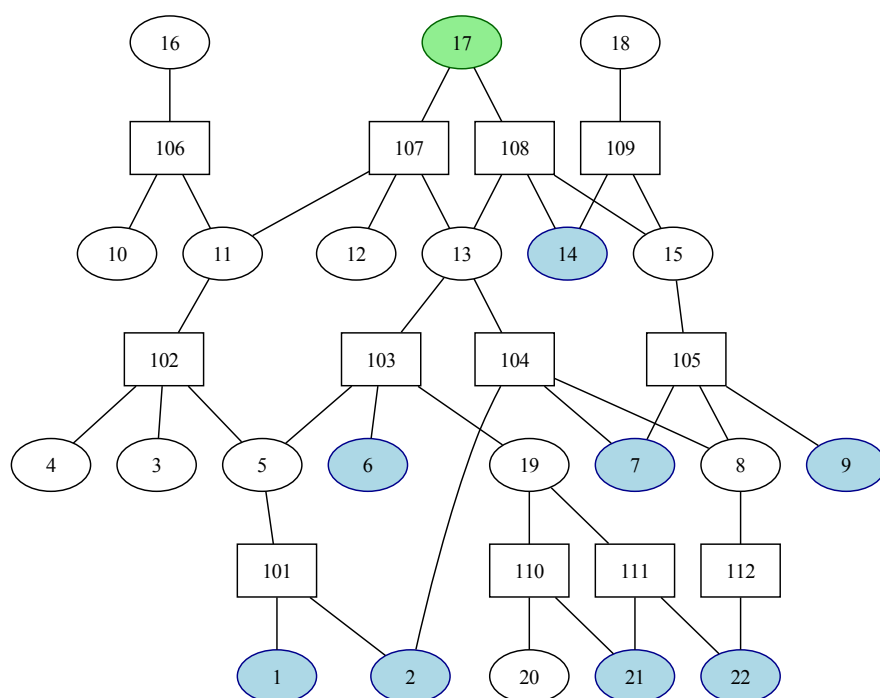


Рисунок 1.1 – Входной граф И/ИЛИ

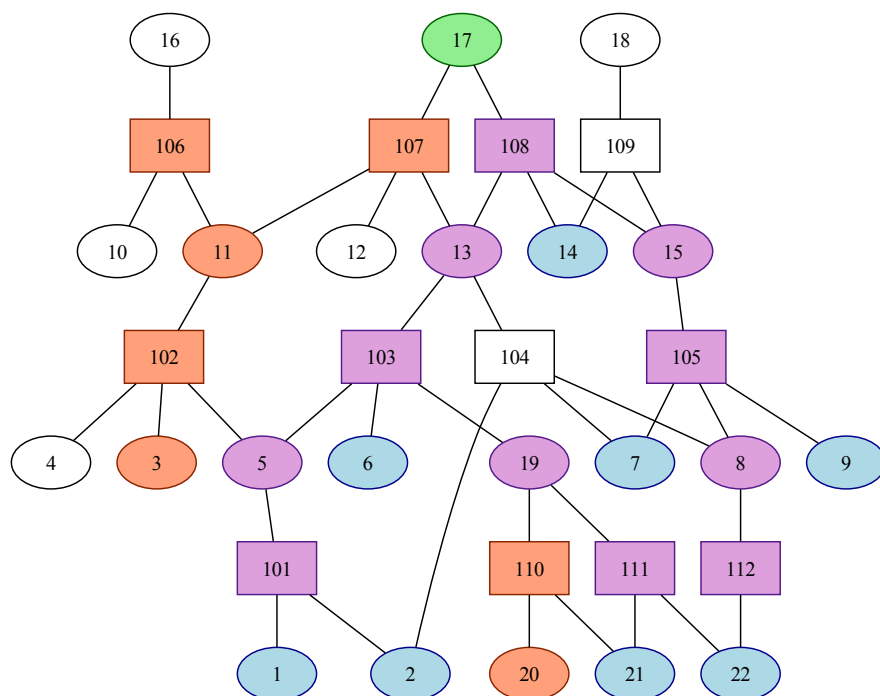


Рисунок 1.2 – Итоговый граф И/ИЛИ с построенным деревом решений

2 Текст программы

В листингах 2.1–2.6 представлен код программы.

Листинг 2.1 — Основной модуль программы

```
1  from data import *
2  from items import Node
3  from search import Search
4  from graph import buildGraph
5
6
7  def main(goal_node: Node, in_node_arr: list[Node]):
8      buildGraph(
9          rules=RULES,
10         goal_node=goal_node,
11         in_node_arr=in_node_arr,
12         graph_name="initial_graph",
13     )
14     search = Search(RULES)
15     search.run(
16         goal_node=goal_node,
17         in_node_arr=in_node_arr.copy(),
18     )
19     buildGraph(
20         rules=RULES,
21         goal_node=goal_node,
22         in_node_arr=in_node_arr,
23         close_node_lst=search.close_node_lst,
24         close_rule_lst=search.close_rule_lst,
25         prohibited_node_lst=search.prohibited_node_lst,
26         prohibited_rule_lst=search.prohibited_rule_lst,
27         graph_name="final_graph",
28     )
29
30
31  if __name__ == "__main__":
32      main(
33          goal_node=NODE_17,
34          in_node_arr=[
35              NODE_1, NODE_2, NODE_6, NODE_7,
36              NODE_9, NODE_14, NODE_21, NODE_22,
37          ],
38      )
```

Листинг 2.2 — Модуль с данными

```
1  from items import Rule, Node
2
3  NODE_1 = Node(1)
4  NODE_2 = Node(2)
5  NODE_3 = Node(3)
6  NODE_4 = Node(4)
7  NODE_5 = Node(5)
8  NODE_6 = Node(6)
9  NODE_7 = Node(7)
10 NODE_8 = Node(8)
11 NODE_9 = Node(9)
12 NODE_10 = Node(10)
13 NODE_11 = Node(11)
14 NODE_12 = Node(12)
15 NODE_13 = Node(13)
16 NODE_14 = Node(14)
17 NODE_15 = Node(15)
18 NODE_16 = Node(16)
19 NODE_17 = Node(17)
20 NODE_18 = Node(18)
21 NODE_19 = Node(19)
22 NODE_20 = Node(20)
23 NODE_21 = Node(21)
24 NODE_22 = Node(22)
25
26 RULES = [
27     Rule(101, NODE_5, [NODE_1, NODE_2]),
28     Rule(102, NODE_11, [NODE_3, NODE_4, NODE_5]),
29     Rule(103, NODE_13, [NODE_5, NODE_6, NODE_19]),
30     Rule(104, NODE_13, [NODE_2, NODE_7, NODE_8]),
31     Rule(105, NODE_15, [NODE_7, NODE_8, NODE_9]),
32     Rule(106, NODE_16, [NODE_10, NODE_11]),
33     Rule(107, NODE_17, [NODE_11, NODE_12, NODE_13]),
34     Rule(108, NODE_17, [NODE_13, NODE_14, NODE_15]),
35     Rule(109, NODE_18, [NODE_14, NODE_15]),
36     Rule(110, NODE_19, [NODE_20, NODE_21]),
37     Rule(111, NODE_19, [NODE_21, NODE_22]),
38     Rule(112, NODE_8, [NODE_22]),
39 ]
```

Листинг 2.3 — Модуль для построения графа

```
1  import graphviz
2
3  from items import Rule, Node
```


Продолжение листинга 2.3

```
4
5
6 def buildGraph(
7     rules: list[Rule],
8     goal_node: Node,
9     in_node_arr: list[Node],
10    graph_name: str,
11    close_node_lst: list[Node] | None = None,
12    close_rule_lst: list[Rule] | None = None,
13    prohibited_node_lst: list[Node] | None = None,
14    prohibited_rule_lst: list[Rule] | None = None,
15 ) -> None:
16     dot = graphviz.Digraph(
17         comment=f'lab_03_{graph_name}'
18     )
19     nodes = []
20     for rule in rules:
21         nodes.append(str(rule.out_node.number))
22         if close_rule_lst and rule in close_rule_lst:
23             dot.node(
24                 name=str(rule.number),
25                 shape="rectangle",
26                 color="purple4",
27                 style="filled",
28                 fillcolor="plum",
29             )
30         elif prohibited_rule_lst and rule in prohibited_rule_lst:
31             dot.node(
32                 name=str(rule.number),
33                 shape="rectangle",
34                 color="orangered4",
35                 style="filled",
36                 fillcolor="lightsalmon",
37             )
38         else:
39             dot.node(
40                 name=str(rule.number),
41                 shape="rectangle",
42             )
43
44     dot.edge(
45         tail_name=str(rule.out_node.number),
46         head_name=str(rule.number),
47         constraint='true',
48         arrowhead="none",
```

Продолжение листинга 2.3

```
49         )
50         for node in rule.node_arr:
51             nodes.append(str(node.number))
52             dot.edge(
53                 tail_name=str(rule.number),
54                 head_name=str(node.number),
55                 constraint='true',
56                 arrowhead="none",
57             )
58     for node in list(dict.fromkeys(nodes)):
59         if node == str(goal_node.number):
60             dot.node(
61                 name=node,
62                 color="darkgreen",
63                 style="filled",
64                 fillcolor="lightgreen",
65             )
66         elif node in [str(in_node.number) for in_node in in_node_arr]:
67             dot.node(
68                 name=node,
69                 color="darkblue",
70                 style="filled",
71                 fillcolor="lightblue",
72             )
73         elif close_node_lst and node in \
74             [str(close_node.number) for close_node in close_node_lst]:
75             dot.node(
76                 name=node,
77                 color="purple4",
78                 style="filled",
79                 fillcolor="plum",
80             )
81         elif prohibited_node_lst and node in \
82             [str(prohibited_node.number) for prohibited_node in
83              ↪ prohibited_node_lst]:
84             dot.node(
85                 name=node,
86                 color="orangered4",
87                 style="filled",
88                 fillcolor="lightsalmon",
89             )
90         else:
91             dot.node(node)
92
93     dot.render(f'./docs/{graph_name}.gv').replace('\\', '/')
```

Листинг 2.4 — Модуль с описанием правил и вершин

```
1  from typing import List
2  from enum import Enum
3
4
5  class Label(Enum):
6      OPEN = 0
7      CLOSE = 1
8      FORBIDDEN = -1
9      VIEWED = 2
10
11
12  class Node:
13      def __init__(self, number: int, flag: int = Label.OPEN):
14          self.number = number
15          self.flag = flag
16
17      def __str__(self):
18          return f'{self.number}'
19
20      def __repr__(self):
21          return f'{self.number}'
22
23
24  class Rule:
25      def __init__(
26          self,
27          number: int,
28          out_node: Node,
29          node_arr: List[Node],
30          label=Label.OPEN
31      ):
32          self.number = number
33          self.out_node = out_node
34          self.node_arr = node_arr # массив входных вершин, связанных связкой И
35          self.label = label # открытое/закрытое/запрещенное
```

Листинг 2.5 — Модуль с реализацией стека

```
1  class Stack:
2      def __init__(self):
3          self.elements = []
4
5      def length(self):
6          return len(self.elements)
7
```

Продолжение листинга 2.5

```
8     def push(self, element):
9         self.elements.append(element)
10
11     def push_arr(self, element_arr):
12         for i in range(len(element_arr) - 1, -1, -1):
13             self.push(element_arr[i])
14
15     def pop(self):
16         if self.length() == 0:
17             return None
18         return self.elements.pop()
19
20     def remove_element(self, element):
21         # чтобы удалить последний обнаруженный элемент в списке
22         self.elements.reverse()
23         try:
24             self.elements.remove(element)
25         except ValueError:
26             pass
27         self.elements.reverse()
28
29     def peek(self):
30         element = self.pop()
31         self.push(element)
32         return element
33
34     def is_exist(self, item):
35         is_found = False
36         tmp_stack = Stack()
37
38         while self.length() != 0:
39             element = self.pop()
40             tmp_stack.push(element)
41
42             if element == item:
43                 is_found = True
44                 break
45
46         while tmp_stack.length() != 0:
47             self.push(tmp_stack.pop())
48
49         return is_found
50
51     def show(self):
52         for element in self.elements:
```

Продолжение листинга 2.5

```
53         print(element, end=' ')
54     print()
```

Листинг 2.6 — Модуль с реализацией обратного поиска в глубину от цели

```
1  from items import Rule, Node, Label
2  from stack import Stack
3
4
5  class Search:
6      """
7      Обратный поиск в глубину от цели
8      """
9      def __init__(self, rule_arr: list[Rule]):
10         self.rule_arr = rule_arr # база знаний
11         self.open_node_st = Stack()
12         self.open_rule_lst = []
13         self.close_node_lst = []
14         self.close_rule_lst = []
15         self.prohibited_node_lst = []
16         self.prohibited_rule_lst = []
17
18         self.goal_node = None
19         self.solution_flg = False
20         self.no_solution_flg = False
21
22     def run(self, goal_node: Node, in_node_arr: list[Node]):
23         self.goal_node = goal_node
24         self.open_node_st.push(goal_node)
25         self.close_node_lst = in_node_arr
26
27         while not self.solution_flg and not self.no_solution_flg:
28             rule_cnt = self.child_search()
29
30             if self.solution_flg:
31                 print('\nРешение было найдено\n')
32                 return
33
34             if rule_cnt == 0 and self.open_node_st.length() < 2:
35                 self.no_solution_flg = True
36                 print("\nРешение не было найдено\n")
37             elif rule_cnt == 0:
38                 print("\nНе было найдено правила, выходной вершиной которого
↪ является текущая вершина")
```

Продолжение листинга 2.6

```
39         print("\nВозвращение назад:")
40         self.backtracking()
41
42     def child_search(self):
43         cnt_rules = 0
44
45         for rule in self.rule_arr:
46             current_node = self.open_node_st.peek()
47             print(f'\nТекущее правило: {rule.number}')
48             print(f'Текущая вершина: {current_node.number}')
49
50             if rule.label != Label.OPEN:
51                 print(f'Правило уже было обработано\n\n{'-' * 64}')
52                 continue
53
54             if rule.out_node == current_node:
55                 print('Выходная вершина правила равна текущей вершине')
56
57                 rule.label = Label.VIEWED
58                 self.open_rule_lst.append(rule)
59
60                 is_new_goal_added = self.add_new_goal(rule.node_arr)
61                 if not is_new_goal_added:
62                     print('Все входные вершины правила являются закрытыми')
63                     print('\nРазметка:')
64                     self.label()
65
66                 cnt_rules += 1
67                 self.print_info()
68                 break
69
70             if self.is_prohibited_node_exist(rule.node_arr):
71                 print('Среди входных вершин правила есть запрещенная')
72                 self.prohibited_rule_lst.append(rule)
73                 rule.label = Label.FORBIDDEN
74
75             self.print_info()
76
77         return cnt_rules
78
79     def label(self):
80         while True:
81             rule = self.open_rule_lst.pop()
82             self.close_rule_lst.append(rule)
83
```

Продолжение листинга 2.6

```
84         node = self.open_node_st.pop()
85         self.close_node_lst.append(node)
86
87         print('    Правило {:3d} -> список закрытых
88         ↪ правил'.format(rule.number))
89
90         print('    Вершина {:3d} -> список закрытых
91         ↪ вершин'.format(node.number))
92
93
94         if node == self.goal_node:
95             self.solution_flg = True
96             break
97
98         current_node = self.open_node_st.peek()
99         current_rule = self.open_rule_lst[-1]
100         if current_rule.out_node != current_node:
101             break
102
103     def backtracking(self):
104         current_goal = self.open_node_st.pop()
105         rule = self.open_rule_lst.pop()
106
107         current_goal.flag = Label.FORBIDDEN
108         self.prohibited_node_lst.append(current_goal)
109
110         rule.label = Label.FORBIDDEN
111         self.prohibited_rule_lst.append(rule)
112
113         print('    Правило {:3d} -> список запрещенных
114         ↪ правил'.format(rule.number))
115
116         print('    Вершина {:3d} -> список запрещенных
117         ↪ вершин'.format(current_goal.number))
118
119         for node in rule.node_arr:
120             print('    Вершина {:3d} должна быть удалена из стека открытых
121             ↪ вершин'.format(node.number))
122             self.open_node_st.remove_element(node)
123
124         print('\n' + '-' * 64)
125
126     def add_new_goal(self, node_arr: list[Node]):
127         new_goal_flg = False
128
129         for node in node_arr[:-1]:
130             if node not in self.close_node_lst:
131                 self.open_node_st.push(node)
```

Продолжение листинга 2.6

```
124         new_goal_flg = True
125
126     return new_goal_flg
127
128     def is_prohibited_node_exist(self, node_arr: list[Node]):
129         for node in node_arr:
130             if node in self.prohibited_node_lst:
131                 return True
132
133         return False
134
135     def print_info(self):
136         print('\nСтек открытых вершин:      ', end='')
137         self.open_node_st.show()
138         print('Список закрытых вершин:      ', end='')
139         self.print_nodes(self.close_node_lst)
140         print('Список запрещенных вершин: ', end='')
141         self.print_nodes(self.prohibited_node_lst)
142
143         print('Список открытых правил:      ', end='')
144         self.print_rules(self.open_rule_lst)
145         print('Список доказанных правил: ', end='')
146         self.print_rules(self.close_rule_lst)
147         print('Список запрещенных правил: ', end='')
148         self.print_rules(self.prohibited_rule_lst)
149
150         print('\n' + '-' * 64)
151
152     def print_nodes(self, node_arr: list[Node]):
153         for node in node_arr:
154             print(node, end=' ')
155         print()
156
157     def print_rules(self, rule_arr: list[Rule]):
158         for rule in rule_arr:
159             print(rule.number, end=' ')
160         print()
```